CMakeLists.txt 指令全参考手册

适用于 VS Code C++ 开发环境 | 现代 CMake 标准 (3.10+)

一、 基础配置指令

这些指令通常出现在文件的最顶部，用于初始化项目环境。

指令	说明与示例
`cmake_minimum_required`	指定所需的最低 CMake 版本。 示例：`cmake_minimum_required(VERSION 3.10)`
`project`	设置项目名称、版本号和语言。 示例：`project(MyProject VERSION 1.0 LANGUAGES CXX)`
`add_executable`	使用指定的源文件生成可执行文件。 示例：`add_executable(app main.cpp src/utils.cpp)`
`add_library`	生成库文件（STATIC, SHARED, 或 MODULE）。 示例：`add_library(mylib STATIC src/lib.cpp)`

二、 目标配置 (Modern CMake)

推荐通过“目标（Target）”来管理属性，而不是使用全局指令。这能有效避免依赖污染。

1. 包含路径与链接库

```
# 添加头文件搜索路径（推荐方式）
target_include_directories(target_name PRIVATE include_dir)

# 链接库文件
target_link_libraries(target_name PRIVATE lib_name)

# 添加宏定义
target_compile_definitions(target_name PRIVATE USE_FEATURE_X)
```

```
# 添加编译选项（如 -Wall）
target_compile_options(target_name PRIVATE -Wall -Wextra)
```

提示： PRIVATE 表示仅当前目标使用；PUBLIC 表示当前目标使用且传播给链接它的目标；INTERFACE 仅传播给链接它的目标。

三、 变量与缓存管理

指令	说明
`set`	定义普通变量或缓存变量。示例：`set(SRC_FILES main.cpp)`
`option`	提供给用户的布尔开关（ON/OFF），可在 VS Code 底部 UI 切换。
`list`	操作列表。示例：`list(APPEND SRC_FILES extra.cpp)`
`file`	文件系统操作。常用：`file(GLOB SOURCES "src/*.cpp")`

四、 流程控制

条件判断

```
if(WIN32)
    message(STATUS "Running on Windows")
elseif(APPLE)
    message(STATUS "Running on macOS")
else()
    message(STATUS "Running on Linux")
endif()
```

循环遍历

```
foreach(item IN LISTS MY_LIST)
    message(STATUS "Item: ${item}")
```

```
endforeach()
```

五、 查找外部依赖

CMake 拥有强大的包查找机制，通过 `find_package` 可以集成 OpenCV, Qt, Boost 等库。

```
# 查找库
find_package(OpenCV REQUIRED)

# 使用查找到的库
target_include_directories(app PRIVATE ${OpenCV_INCLUDE_DIRS})
target_link_libraries(app PRIVATE ${OpenCV_LIBS})
```

六、 常用内置变量索引

变量名	用途
`PROJECT_SOURCE_DIR`	当前项目的根目录路径。
`CMAKE_BUILD_TYPE`	构建类型：Debug, Release, RelWithDebInfo, MinSizeRel。
`CMAKE_CXX_STANDARD`	设置 C++ 标准（如 11, 14, 17, 20）。
`EXECUTABLE_OUTPUT_PATH`	设置生成的可执行文件存放目录。

七、 VS Code 与 CMake 协作建议

- **自动配置：** 每次修改 `CMakeLists.txt` 后，按 `Ctrl+Shift+P` 运行 `CMake: Configure`。
- **多目录管理：** 使用 `add_subdirectory(subdir)` 将大型项目拆分为多个模块，每个子目录自持 `CMakeLists.txt`。
- **调试信息：** 使用 `message(STATUS "Text...")` 指令可以在 VS Code 的“输出”窗口（Output -> CMake/Build）看到调试打印。